Lista de inicialización

Inicializa los miembros antes de ejecutar el cuerpo

C++20 • La lista construye subobjetos; el cuerpo opera sobre ellos.

Declaraciones relevantes

La clase `Circle` tiene un miembro de datos, un miembro protegido y un constructor.

```
class Circle : public Shape
{
private:
    double radio;

protected:
    double factorEscala;

public:
    Circle(unsigned int id, double r);
};
```

Lista de inicialización

La lista construye la base y los miembros antes de ejecutar el cuerpo del constructor.

```
Circle::Circle(unsigned id, double r)
    : Shape(id, "Círculo"),
      radio(r > 0.0 ? r : 1.0),
      factorEscala(1.0)
{
    // Aquí empieza el cuerpo.
}
```

Orden real en este ejemplo

Las bases se construyen antes que los miembros. Los miembros siguen su orden de declaración, aunque cambies el orden escrito en la lista.

1. `Shape(id, "Círculo")`
2. `radio`
3. `factorEscala`
4. `Cuerpo del constructor`

Asignar en el cuerpo es distinto

Aquí los `double` sin inicializador previo reciben su valor mediante asignación. No deben leerse antes de ella. Un miembro `string` o `vector` ya habría sido construido antes de entrar al cuerpo.

```
Circle::Circle(unsigned id, double r)
    : Shape(id, "Círculo")
{
    radio = r > 0.0 ? r : 1.0;
    factorEscala = 1.0;
}
```

Cuándo hace falta inicializar

Las referencias deben enlazarse al construirse. Los miembros `const` necesitan inicialización y pueden tener un inicializador en su declaración. Para pasar argumentos al constructor de la base, usa la lista.